

Ruta7 - Car Rental Platform

Assignment 1 - Report

My project repository : <https://github.com/Rdrg-Blnc-IE/devops-assignment-1>

1. SDLC Model

Model chosen

This project followed a lightweight, iterative-incremental SDLC: each working session targeted one small, demonstrable slice of the system (one domain, one builder, one route, one page), validated it (manual run, or a pytest file), and committed it before moving to the next slice. This is closest to a self-managed Kanban-style flow rather than fixed-length Scrum sprints.

This model was chosen for three reasons specific to this project: it is a solo project, so sprint ceremonies (standups, planning poker) have no audience; the assignment's own structure (two independent feature domains, SQLite, a web layer) decomposes naturally into small vertical slices; and the explicit requirement for an ADR log spanning multiple distinct dates rewards working in small, dated increments rather than one large batch.

SMART goals

Letter	Criterion	This project's goal
S	Specific	Implement the Vehicle domain (Vehicle, VehicleBuilder, validation rules) and the Reservations domain (Rental, RentalBuilder, availability and pricing logic) as two independently testable modules.
M	Measurable	$\geq 70\%$ automated test coverage on the core business logic of both domains, measured with pytest-cov.
A	Achievable	Each domain's builder pattern (with <code>_x()</code> chaining + <code>build()</code> validation) was scoped to what one person can design, implement, and explain cold within the assignment window.
R	Relevant	Both goals map directly to the assignment's required deliverables: §3 (two domains), §4 (coverage threshold), §5 (ADR/AI usage log).
T	Time-bound	Vehicle domain complete and tested before Reservations began; web layer (landing page + customer flow) complete before the report was drafted; enterprise web layer deferred past the point where time ran out (see deviation below).

How practice diverged from the plan

The plan mostly held for the backend: Vehicle (`vehicle_info.py`), Reservations (`rentals.py`), and the Transport module (`transport.py`, Factory Method pattern for truck/ship logistics) were each built, debugged, and unit-tested as complete, independent slices before the next one started.

Two deviations are worth naming honestly, since they are exactly what the ADR-5 entry ("one thing deliberately not built") and the comprehension check are designed to surface:

- Tests were written after each builder's validation logic, not before it - a pragmatic, not strict-TDD, approach. Each round of test-writing also surfaced real bugs in the already-written code (a positional-argument bug in Vehicle construction, a singleton/instance-access bug in the DB layer, a table-name mismatch between the schema and the DB class), which the tests then caught rather than prevented.
- The enterprise side of the web app (new vehicle, vehicle list with filters, transport request, transport queue view, rentals list, customers list - six of the eight planned screens) was scaffolded as a single placeholder route and never built out. Time was spent instead on getting the customer-facing flow (login, dashboard, new rental, rental detail) fully working end to end against the real database. This is the clearest example of the iterative plan being followed - ship one working vertical slice completely - at the cost of overall feature breadth.

2. Architecture Overview

The application is a single Flask process (python app.py), matching the single-process constraint. A browser talks to Flask over HTTP; Flask's app factory (create_app()) reads configuration once from environment variables and registers two blueprints, one per audience (customer, enterprise). Each blueprint calls into the domain layer - plain Python classes with no Flask dependency - which in turn goes through one shared DB instance to SQLite.

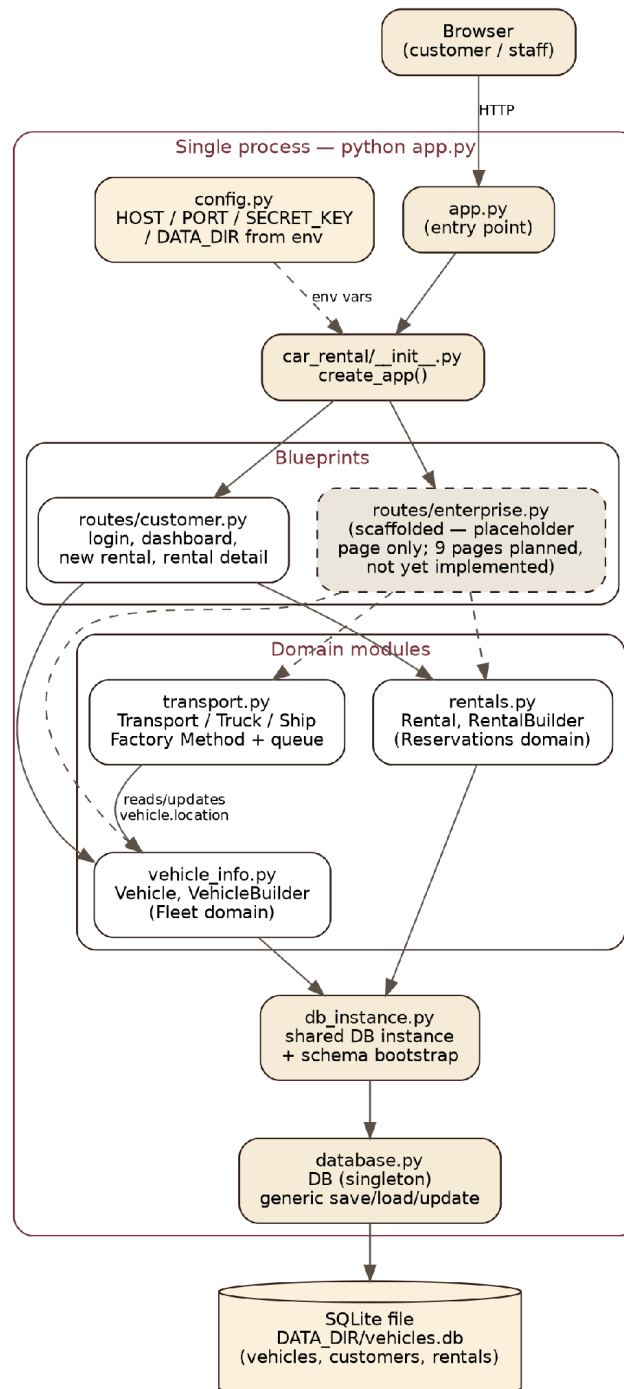


Figure 1. Architecture overview - single-process Flask app, two blueprints, three domain modules, one shared SQLite connection.

Two points this diagram makes explicit, since they matter for the deployment contract in Assignment 2: there is exactly one process and one dependency manifest; and the SQLite file's location is read from one environment variable (DATA_DIR) with a working default, not hard-coded.

The enterprise blueprint is shown with a dashed border because it is currently a single placeholder route, not the eight planned screens - this matches the actual repository, not the original plan, per the deviation noted in Section 1.

3. Database Model

All persistent state lives in three SQLite tables, created idempotently at startup by db_instance.py (CREATE TABLE IF NOT EXISTS, safe to run on every boot). This matches ADR-3's data-model decision: Reservations never reads Vehicle's table directly - a rental refers to a vehicle only by its reg_num foreign key, and to a customer only by its id foreign key, keeping the two feature domains loosely coupled while still relationally consistent.

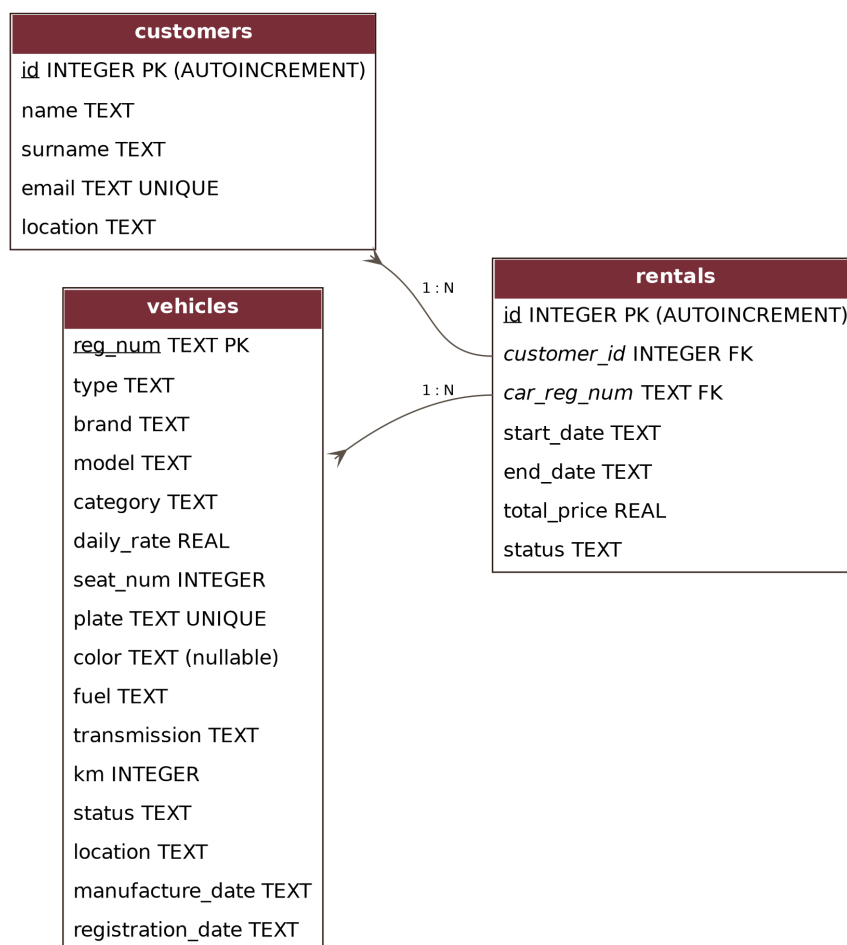


Figure 2. Database schema - vehicles and customers are both referenced by rentals; vehicles and customers have no direct relationship to each other.

vehicles.reg_num is a natural-key primary key (the vehicle's registration/VIN string), not an autoincrement integer, since it is already guaranteed unique in the real world. customers.id and rentals.id are autoincrement integers, since neither has a natural external identifier. rentals.status and vehicles.status are plain TEXT columns backed by Python enums (RentalStatus, VehicleStatus) at the application layer - SQLite has no native enum type, so validity is enforced in Python before a row is ever written.

4. README

The following is the setup section as it appears in the repository's README.md, included here per §8.

Requirements

- Python 3.11+
- uv

Setup

This repo uses uv (<https://docs.astral.sh/uv/>) to manage the Python environment. Install uv once per machine (see uv's docs):

```
git clone https://github.com/Rdrg-Blnc-IE/devops-assignment-1
cd devops-assignment-1

uv venv                                # create a local virtual environment (.venv)
uv pip install -r requirements.txt     # install pytest into it
```

Running the app

No manual migration step is needed - db_instance.py creates the SQLite schema automatically on first run if it does not already exist.

```
Uv run python app.py
# then open http://localhost:5000
```

Configuration is entirely via environment variables, all with working defaults:

- PORT - port to listen on (default 5000)
- HOST - bind address (default 0.0.0.0)
- DATA_DIR - directory for the SQLite file (default ./src)
- SECRET_KEY - Flask session signing key (default is a dev-only placeholder; set a real value for any shared deployment)

Seeding sample data (optional)

A one-time script transforms a public used-car dataset into realistic seed data (vehicles, customers, rentals) for local development and demoing:

```
python car_rental/data_manipulation.py
```

Running tests and coverage

Automated unit tests cover the core business logic of both domains (validation rules in VehicleBuilder.build() and RentalBuilder.build(), the transport queue/routing logic, and the DB layer's CRUD operations):

```
pytest tests/ --cov=car_rental --cov-report=term-missing
```

5. AI-Disclosure Statement

I acknowledge the use of Claude (Anthropic) to design, implement, debug, and document sections of this project, across architecture decisions, backend domain logic, database design, automated tests, and the web front end. This report was also started with AI, then completed manually.

The output of these prompts was used as a starting implementation in every case, which I then ran, tested, and in several instances had to correct after AI-introduced or AI-missed bugs surfaced during testing (for example, a positional-argument ordering bug in vehicle construction, and a table-name mismatch between the database schema and the database access layer). Accepted code was adapted to fit the project's existing naming and structure before being committed. A complete, prompt-by-prompt log with disposition (accepted / modified / rejected) and an explanation in my own words of how each accepted piece of code works is maintained in `AI_USAGE.md`, per the course syllabus.